

Функции

Создавать функции с параметрами и return, отделять вычисление от вывода и понимать область видимости.

DAY 14

FUNCTION

parameter

argument

return

scope

ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

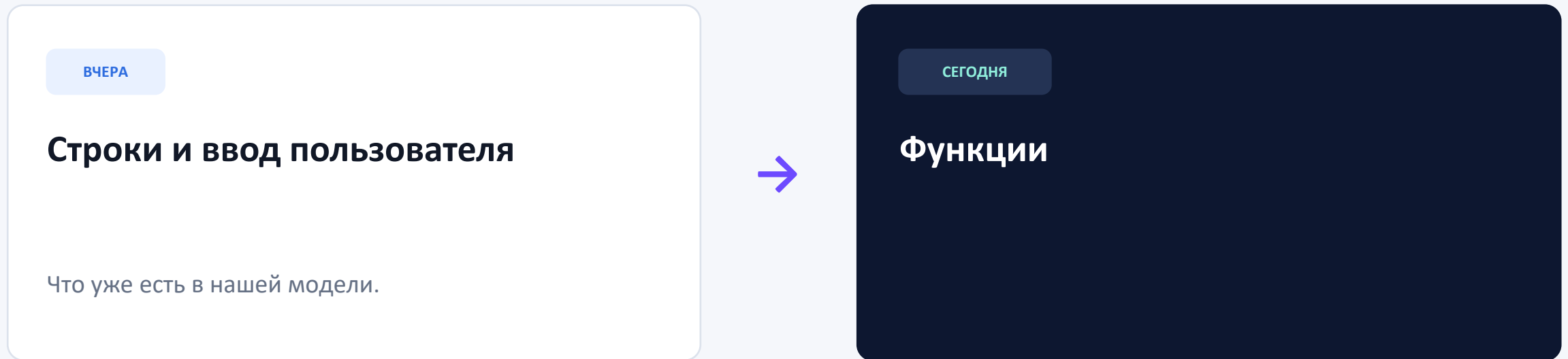
Конкретные действия вместо списка терминов.

- 01 Создавать функции с параметрами и return, отделять вычисление от вывода и понимать область видимости.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример кода и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.



Вчера: Строки и ввод пользователя. Сегодня мы добавляем следующий слой: функции.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Функции убирают повторения, дают коду понятные имена и позволяют тестировать отдельные кусочки программы.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

Функция — кофемашина: есть имя действия, входные параметры, внутренние шаги и результат, который можно вернуть наружу.

ПЕРЕВОД В IT

function	именованный переиспользуемый блок кода
parameter	имя входного значения в определении функции
argument	конкретное значение, переданное при вызове

Слой 1: function • parameter • argument

Сначала смысл, затем профессиональный термин.

**function**

именованный переиспользуемый блок кода

**parameter**

имя входного значения в определении функции

**argument**

конкретное значение, переданное при вызове

Слой 2: return • scope • docstring

Сначала смысл, затем профессиональный термин.

**return**

оператор возврата результата из функции

**scope**

область программы, где доступно имя

**docstring**

строка документации внутри функции, класса или модуля

Слой 3: call • default argument

Сначала смысл, затем профессиональный термин.

call

запуск функции с указанными аргументами

default argument

значение параметра, используемое без явного аргумента

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: keyword argument • pure function

Сначала смысл, затем профессиональный термин.

keyword argument

аргумент, переданный с именем параметра

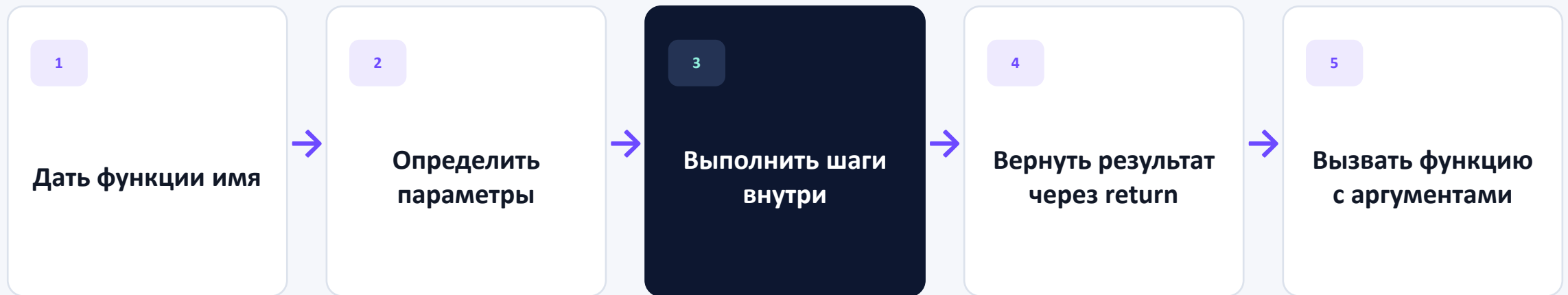
pure function

функция без побочных эффектов при одинаковом вводе

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day14_step1

```
def calculate_total(price, quantity):  
    return price * quantity  
  
result = calculate_total(120, 3)  
print(result)
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

```
def calculate_total(price, quantity):
```

Создаём функцию и даём ей имя; код внутри выполнится при вызове.

```
    return price * quantity
```

Python выполняет эту строку сверху вниз.

```
result = calculate_total(120, 3)
```

Сохраняем значение или результат вычисления под понятным именем.

```
print(result)
```

Показываем результат, чтобы увидеть состояние программы.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
def calculate_total(price, quantity):  
    return price * quantity  
  
result = calculate_total(120, 3)  
print(result)
```

РЕЗУЛЬТАТ

360

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.

```
def greet(name="друг"):
    return f"Привет, {name}!"

print(greet())
print(greet("Anna"))
```

changed



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

Привет, друг!
Привет, Anna!

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.



wrong

```
def add(a, b):  
    print(a + b)  
  
result = add(2, 3)  
print(result)
```

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

Функция печатает 5, но ничего не возвращает. Поэтому result получает None. Для вычисления нужен return.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

ИСПРАВЛЕНИЕ

Исправленный вариант

Меняем только то, что действительно было причиной.



fixed

```
def add(a, b):  
    return a + b  
  
result = add(2, 3)  
print(result)
```

ПРАВИЛО

**Не переписывай всё
решение. Найди
минимальное
осмысленное
исправление.**

Что изменилось после исправления?

Ответь без запуска.

1

Исчезла ли исходная причина проблемы?

2

Какой результат/поведение теперь ожидаем?

3

Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Написать функцию `apply_discount(price, percent)`, которая возвращает цену после скидки.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 Получаем цену и процент как параметры.
- 2 Переводим процент в долю.
- 3 Вычисляем размер скидки.
- 4 Вычитаем его из цены.
- 5 Возвращаем результат, а не печатаем внутри.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.

```
def apply_discount(price, percent):  
    discount = price * percent / 100  
    return price - discount
```

solution_part1

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



```
def apply_discount(price, percent):  
    discount = price * percent / 100  
    return price - discount  
  
print(apply_discount(200, 15))
```

solution_final

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

170.0

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобранную — но теперь первый ход за тобой.

Создай функцию `format_user(name, age)`, которая возвращает строку "Anna — 25 лет".

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

Результат удобно собрать f-строкой и вернуть через return.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



```
def format_user(name, age):  
    return f"{name} — {age} лет"  
  
print(format_user("Anna", 25))
```

practice_solution

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

Anna — 25 лет

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую законченную задачу.

ЗАДАНИЕ

Сделай функцию `is_adult(age)`, которая возвращает `True` или `False`.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.

A dark-themed terminal window with a title bar at the top containing three colored circles (red, yellow, green) on the left and the text 'mini_project' on the right. The terminal contains Python code:

```
def is_adult(age):  
    return age >= 18  
  
print(is_adult(20))  
print(is_adult(16))
```

```
def is_adult(age):  
    return age >= 18  
  
print(is_adult(20))  
print(is_adult(16))
```

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

True
False

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01

Путать параметр и аргумент.

02

Использовать `print()` вместо `return`.

03

Делать функцию слишком большой и разнородной.

04

Полагаться на глобальные переменные без необходимости.

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «function» и какую проблему это понятие помогает описать?

2

Чем «function» отличается от «return»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 **function — именованный переиспользуемый блок кода.**

2 **function: именованный переиспользуемый блок кода. return: оператор возврата результата из функции.**

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3 **360**

4 **Функция печатает 5, но ничего не возвращает. Поэтому result получает None. Для вычисления нужен return.**

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ **Создавать функции с параметрами и `return`, отделять вычисление от вывода и понимать область видимости.**
- ✓ Читать и объяснять пример темы «Функции».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

[ДАЛЬШЕ](#)

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Ошибки и отладка

Завтра: Ошибки и отладка. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.